

DATA STRUCTURE

Graded lab Tasks 02



SEPTEMBER 21, 2026

INZA BIBI
FA25-BCS-031

LAB TASK 1

CODE

Reverse Display (Loop + Recursion)

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {
```

```
    int data;
```

```
    Node* next;
```

```
};
```

```
Node* head = NULL;
```

```
// Insert at end
```

```
void insertEnd(int value) {
```

```
    Node* newNode = new Node;
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
    if (head == NULL) {
```

```
        head = newNode;
```

```
        return;
```

```
    }
```

```
    Node* temp = head;
```

```
    while (temp->next != NULL) {
```

```

        temp = temp->next;
    }
    temp->next = newNode;
}

// Normal display
void display(Node* h) {
    Node* temp = h;
    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

// Reverse using Loop (array)
void displayReverseLoop() {
    int arr[100];
    int count = 0;

    Node* temp = head;
    while (temp != NULL) {
        arr[count] = temp->data;
        count++;
        temp = temp->next;
    }
}

```

```

cout << "Reverse (Loop): ";
for (int i = count - 1; i >= 0; i--) {
    cout << arr[i] << " -> ";
}
cout << "NULL" << endl;
}

// Reverse using Recursion
void displayReverseRecursion(Node* temp) {
    if (temp == NULL) {
        return;
    }
    displayReverseRecursion(temp->next);
    cout << temp->data << " -> ";
}

int main() {
    insertEnd(10);
    insertEnd(20);
    insertEnd(30);
    insertEnd(40);

    cout << "Original List: ";
    display(head);

```

```

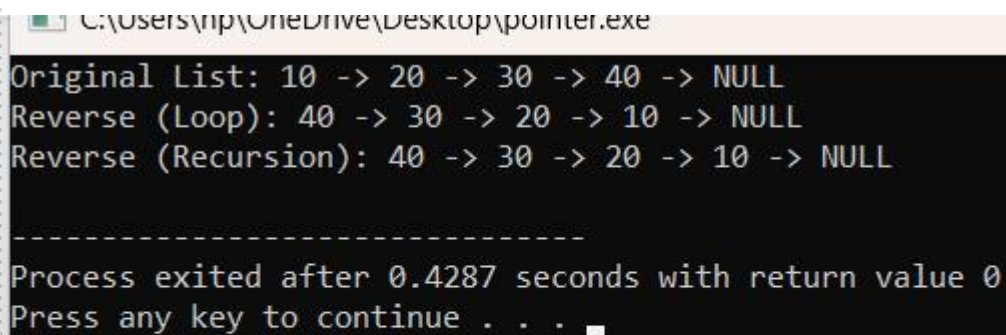
displayReverseLoop();

cout << "Reverse (Recursion): ";
displayReverseRecursion(head);

cout << "NULL" << endl;

return 0;
}

```



```

C:\Users\hp\OneDrive\Desktop\pointer.exe
Original List: 10 -> 20 -> 30 -> 40 -> NULL
Reverse (Loop): 40 -> 30 -> 20 -> 10 -> NULL
Reverse (Recursion): 40 -> 30 -> 20 -> 10 -> NULL

-----
Process exited after 0.4287 seconds with return value 0
Press any key to continue . . .

```

PROGRAM 2: Merge Two Linked Lists

```

#include <iostream>

using namespace std;

struct Node {
    int data;
    Node* next;
}

```

```
};

// Insert at end
void insertEnd(Node* &head, int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }

    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

// Display
void display(Node* h) {
    Node* temp = h;
    while (temp != NULL) {
```

```

        cout << temp->data << " -> ";

        temp = temp->next;
    }

    cout << "NULL" << endl;
}

// Merge two lists into a new list
Node* mergeLists(Node* list1, Node* list2) {
    Node* newHead = NULL;

    Node* newTail = NULL;

    // Copy list1
    Node* temp = list1;
    while (temp != NULL) {
        Node* newNode = new Node;

        newNode->data = temp->data;
        newNode->next = NULL;

        if (newHead == NULL) {
            newHead = newNode;
            newTail = newNode;
        } else {
            newTail->next = newNode;
            newTail = newNode;
        }
    }
}

```

```
}  
  
temp = temp->next;  
  
}  
  
// Copy list2  
temp = list2;  
while (temp != NULL) {  
    Node* newNode = new Node;  
    newNode->data = temp->data;  
    newNode->next = NULL;  
  
    if (newHead == NULL) {  
        newHead = newNode;  
        newTail = newNode;  
    } else {  
        newTail->next = newNode;  
        newTail = newNode;  
    }  
  
    temp = temp->next;  
}  
  
return newHead;  
}
```

```
int main() {  
    Node* list1 = NULL;  
    Node* list2 = NULL;  
  
    insertEnd(list1, 1);  
    insertEnd(list1, 3);  
    insertEnd(list1, 5);  
  
    insertEnd(list2, 2);  
    insertEnd(list2, 4);  
    insertEnd(list2, 6);  
  
    cout << "List 1: ";  
    display(list1);  
  
    cout << "List 2: ";  
    display(list2);  
  
    Node* merged = mergeLists(list1, list2);  
  
    cout << "Merged List: ";  
    display(merged);  
  
    return 0;  
}
```

```
}
```

```
C:\Users\hp\OneDrive\Desktop\pointer.exe
```

```
List 1: 1 -> 3 -> 5 -> NULL
```

```
List 2: 2 -> 4 -> 6 -> NULL
```

```
Merged List: 1 -> 3 -> 5 -> 2 -> 4 -> 6 -> NULL
```

```
-----
```

```
Process exited after 0.3731 seconds with return value 0
```

```
Press any key to continue . . .
```

PROGRAM 3: Find Multiple Occurrences

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {
```

```
    int data;
```

```
    Node* next;
```

```
};
```

```
Node* head = NULL;
```

```
// Insert at end
```

```
void insertEnd(int value) {
```

```
    Node* newNode = new Node;
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
if (head == NULL) {  
    head = newNode;  
    return;  
}
```

```
Node* temp = head;  
while (temp->next != NULL) {  
    temp = temp->next;  
}  
temp->next = newNode;  
}
```

```
// Display
```

```
void display(Node* h) {  
    Node* temp = h;  
    while (temp != NULL) {  
        cout << temp->data << " -> ";  
        temp = temp->next;  
    }  
    cout << "NULL" << endl;  
}
```

```
// Find all occurrences
```

```

void findOccurrences(Node* head, int value) {
    Node* temp = head;
    int position = 0;
    int count = 0;
    bool found = false;

    cout << "Value " << value << " found at positions: ";

    while (temp != NULL) {
        if (temp->data == value) {
            cout << position << " ";
            count++;
            found = true;
        }
        position++;
        temp = temp->next;
    }

    if (!found) {
        cout << "Not found";
    } else {
        cout << "\nTotal occurrences: " << count;
    }

    cout << endl;
}

```

```
}  
  
int main() {  
    insertEnd(10);  
    insertEnd(20);  
    insertEnd(10);  
    insertEnd(30);  
    insertEnd(10);  
    insertEnd(40);  
  
    cout << "List: ";  
    display(head);  
  
    findOccurrences(head, 10);  
    findOccurrences(head, 20);  
    findOccurrences(head, 99);  
  
    return 0;  
}
```

```
List: 10 -> 20 -> 10 -> 30 -> 10 -> 40 -> NULL
```

```
Value 10 found at positions: 0 2 4
```

```
Total occurrences: 3
```

```
Value 20 found at positions: 1
```

```
Total occurrences: 1
```

```
Value 99 found at positions: Not found
```

```
-----
```

```
Process exited after 0.4886 seconds with return value 0
```

```
Press any key to continue . . .
```